SENAI • PROGRAMAÇÃO DE APLICATIVOS

Cantina MetalVale:
lanche sem ficha de papel

Como o sistema funciona por dentro — API em PHP e QR Code

Programação de Aplicativos • Projeto Cantina • HTML + CSS + JavaScript + PHP + MySQL
Código: github.com/wellingtonsenai/aplicativo

VISÃO GERAL

Da ficha de papel ao QR Code

Antes: ficha de papel

A ficha se perde, é copiada, é usada duas vezes. No fim do dia ninguém sabe quantos lanches saíram nem para qual turma.

Agora: QR Code do aluno

Cada aluno tem um QR único. No balcão, o atendente aponta a câmera e o sistema responde na hora: **entregue** ou **não liberado**.

E o histórico fica

Toda entrega é gravada com aluno, atendente, dia e hora. A supervisão tira o relatório por dia e por turma.

A REGRA PRINCIPAL

Um lanche por aluno por dia. Quem garante isso não é a tela — é o banco de dados, com `UNIQUE (aluno_id, data)`.

VISÃO GERAL

As telas e quem usa cada uma

Tela	Para que serve	Atendente	Supervisor
`index.html`	Login com matrícula e senha — depois vai direto para o lanche	sim	sim
`lanche.html`	Lê o QR pela câmera e dá baixa na entrega	sim	sim
`entregues.html`	Relatório de entregues, filtrando por dia e turma	não	sim
`alunos.html`	Cadastro de alunos e download do QR Code	não	sim
`turmas.html`	Cadastro de turmas (ex.: Desenvolvimento de Sistemas V1 22255)	não	sim
`usuarios.html`	Cadastro de atendentes e supervisores	não	sim

Esconder o link no menu é só visual. Quem **bloqueia** de verdade é o PHP: o atendente que digitar `alunos.html` na mão recebe `401` da API.

VISÃO GERAL

As três camadas do sistema

HTML

Navegador

Telas, CSS e JavaScript. Mostra os dados e coleta o que o usuário digita.

fetch()

Pedido HTTP

O JavaScript pede ou envia dados em **JSON**, sem recarregar a página.

PHP

API no servidor

Confere login e perfil, valida, aplica as regras e fala com o banco.

MySQL

Banco de dados

Guarda turmas, alunos, usuários e entregas. Garante as regras com chaves.

A IDEIA CENTRAL

O navegador **nunca** fala direto com o MySQL. Ele só conversa com a API — e a API decide o que pode.

api/

A API

Os arquivos PHP que fazem o CRUD no MySQL e devolvem JSON

Um arquivo PHP para cada assunto

Arquivo	GET (buscar)	POST (enviar)	Quem pode
`login.php`	—	confere matrícula e senha, abre a sessão	qualquer um
`logout.php`	encerra a sessão	—	logado
`entregas.php`	relatório por dia e turma	dá baixa no lanche pelo código do QR	GET: supervisor • POST: logado
`alunos.php`	lista os alunos	`acao` : inserir, editar ou excluir	supervisor
`turmas.php`	lista as turmas	`acao` : inserir, editar ou excluir	supervisor
`usuarios.php`	lista os usuários	`acao` : inserir, editar ou excluir	supervisor
`conexao.php`	não é chamado pela tela: é incluído por todos os outros (`require 'conexao.php'`)		

> Toda resposta é **JSON**: `{"ok": true}` quando deu certo, `{"erro": "..."}` quando não deu — junto com o código HTTP (`400` , `401` , `409`).

Uma chamada de ponta a ponta: o login

JS/FUNCOES.JS — NAVEGADOR

```
async function autenticar(matricula, senha) {
  const resposta = await fetch('api/login.php', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ matricula, senha })
  });
  return await resposta.json();
}
```

API/LOGIN.PHP — SERVIDOR

```
require 'conexao.php';
$dados = lerCorpo();

$stmt = $pdo->prepare('SELECT ... FROM usuarios
                       WHERE matricula = ? AND ativo = 1');
$stmt->execute([$dados['matricula']]);
$usuario = $stmt->fetch();

if (!$usuario || !password_verify($dados['senha'], $usuario['senha'])) {
    responder(['aceito' => false, ...]);
}
$_SESSION['usuario'] = $usuario;  // lembra quem logou
```

```
// resposta que volta para o JavaScript
{"aceito":true,"mensagem":"Bem-vindo(a), Carla Mendes!","usuario":{"id":"3","matricula":"2001","nome":"Carla Mendes","perfil":"supervisor"}}
```

O que todo arquivo da API reaproveita

API/CONEXAO.PHP (RESUMO)

```php
header('Content-Type: application/json; charset=utf-8');

function responder($dados, $status = 200) {
    http_response_code($status);
    echo json_encode($dados);
    exit;
}

$pdo = new PDO('mysql:host=localhost;dbname=cantina', 'root', '');
session_start();

function exigirSupervisor() {
    if (($_SESSION['usuario']['perfil'] ?? '') !== 'supervisor') {
        responder(['erro' => 'Acesso permitido só para supervisor.'], 401);
    }
}
```

NUNCA: VALOR DO USUÁRIO COLADO NO SQL

```php
$pdo->query("DELETE FROM alunos WHERE id = $id");
// $id = "1 OR 1=1"  ->  apaga TODOS os alunos
```

SEMPRE: PREPARED STATEMENT COM ?

```php
$pdo->prepare('DELETE FROM alunos WHERE id = ?')
    ->execute([$id]);
```

POR QUE ISSO IMPORTA

O `?` faz o MySQL tratar o valor como **dado**, nunca como comando. É a defesa contra **SQL Injection**.

PHP ≠ JS?

Por que PHP e não tudo em JavaScript?

A primeira versão do sistema era só JavaScript — e funcionava. Qual era o problema?

A primeira versão: senhas dentro do JavaScript

JS/USUARIOS.JS (VERSÃO ANTIGA, JÁ APAGADA)

```
let usuarios = [
  { matricula: "1001", nome: "Ana Souza",
    senha: "cantina123", perfil: "atendente" },
  { matricula: "2001", nome: "Carla Mendes",
    senha: "super2026", perfil: "supervisor" }
];

// Login comparava no próprio navegador
if (usuario.senha !== senha) { ... }
```

```
// qualquer aluno, no F12 > Console:
> usuarios
[{matricula:"2001", nome:"Carla Mendes", senha:"super2026", ...}]
```

O NAVEGADOR É DO USUÁRIO

Tudo que vai para o navegador — HTML, CSS, **JavaScript e os dados dentro dele** — pode ser lido e alterado por quem está usando.

Hoje a senha só existe no banco, como **hash** (`password_hash()`), e só o PHP confere (`password_verify()`).

Só JavaScript × JavaScript + API PHP

Tudo no navegador

- Senhas e dados visíveis no `F12`
- Não consegue falar com o MySQL: o navegador só fala HTTP
- Usuário e senha do banco teriam que ir no código
- Regras burláveis: o aluno muda o JS e libera 2 lanches
- Dados só naquele computador (`localStorage`)
- Recarregou a página, as alterações somem

Navegador + API PHP

- Senha do banco fica no servidor, em `conexao.php`
- PHP fala com o MySQL pelo PDO
- Login com **sessão**: o servidor sabe quem é quem
- Regras no servidor e no banco: não dá para burlar pelo F12
- Todos os atendentes veem os mesmos dados
- Tudo gravado no banco, com histórico

Então o JavaScript não valida nada?

🙂

Validação no JavaScript = conforto

Avisa na hora que falta um campo, sem ir ao servidor. Melhora a experiência — mas pode ser desligada por quem sabe abrir o F12.

```
if (dados.nome === '') mostrarMensagem(...)
```

🔒

Validação no PHP = segurança

Repete as checagens no servidor, onde o usuário não mexe: campos vazios, perfil válido, matrícula repetida, quem está logado.

```
if (!in_array($perfil, ['atendente','supervisor']))
```

E POR QUE PHP, E NÃO NODE.JS?

O ponto é **ter um servidor**, não a linguagem. Node.js também resolveria. Escolhemos PHP porque o **WAMP** já traz Apache + PHP + MySQL prontos no laboratório.

QR

Geração do QR Code

Do cadastro do aluno até a baixa no balcão

O que vai dentro do QR — e como vira imagem

1

PHP sorteia o código

No cadastro do aluno: 32 caracteres aleatórios.

2

Banco guarda

Coluna `codigo` com `UNIQUE`: nunca se repete.

3

qrcodejs desenha

A biblioteca transforma o código em quadradinhos num `canvas`.

4

Vira PNG

Nome + turma + QR num só canvas, baixado pelo botão QR.

API/ALUNOS.PHP — AO INSERIR

```php
$pdo->prepare('INSERT INTO alunos (matricula, nome, turma_id, codigo, ativo)
               VALUES (?, ?, ?, ?, ?)')
    ->execute([$matricula, $nome, $turmaId,
               bin2hex(random_bytes(16)), $ativo]);

// ex.: 714cd53eff57870fdfafc4ab09621a09
```

ALUNOS.HTML — BOTÃO QR

```javascript
const area = document.createElement('div');  // fora da tela
new QRCode(area, { text: aluno.codigo, width: 220, height: 220 });
const qr = area.querySelector('canvas');

// novo canvas: escreve nome e turma, desenha o QR embaixo
ctx.fillText(aluno.nome, margem, margem);
ctx.drawImage(qr, x, topoQr);

link.href = canvas.toDataURL('image/png');
link.download = 'qrcode-' + aluno.matricula + '.png';
```

Por que não colocar a matrícula no QR? Matrícula é fácil de adivinhar — qualquer um geraria o QR do colega. O código aleatório não dá para inventar.

No balcão: ler o QR e dar baixa

LANCHE.HTML — CÂMERA (BIBLIOTECA HTML5-QRCODE)

```
const leitor = new Html5Qrcode('leitor');
leitor.start({ facingMode: 'environment' },
             { fps: 10, qrbox: 220 }, darBaixa);

async function darBaixa(codigo) {
    leitor.stop();          // desliga a câmera
    const resposta = await fetch('api/entregas.php', {
        method: 'POST',
        body: JSON.stringify({ codigo })
    });
    // verde: "Lanche entregue" | vermelho: "NÃO liberado"
}
```

API/ENTREGAS.PHP — A BAIXA

```
// 1. o código existe e o aluno está ativo?
SELECT ... FROM alunos WHERE codigo = ? AND ativo = 1

// 2. tenta gravar a entrega de hoje
INSERT INTO entregas (aluno_id, usuario_id, data)
VALUES (?, ?, CURDATE())

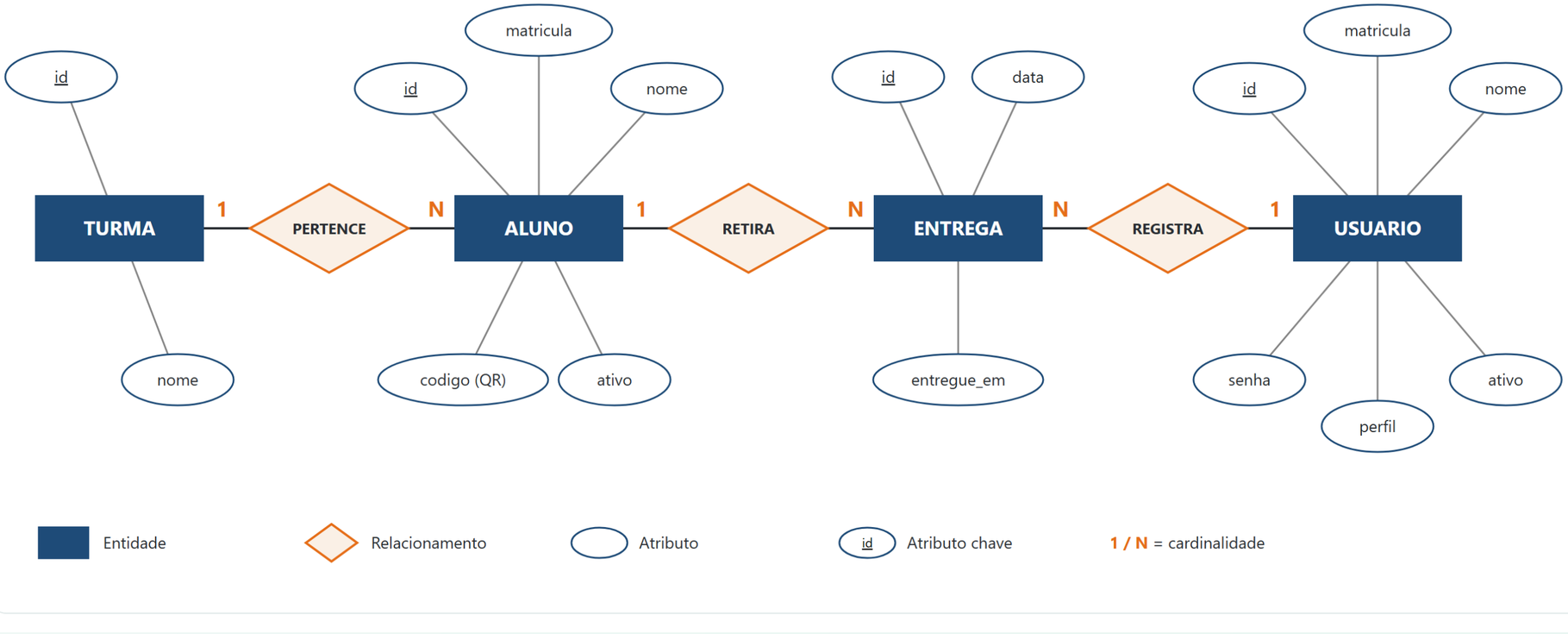
// 3. se já existe (aluno_id, data): erro 23000
catch (PDOException $e) {
    responder(['erro' => '... já retirou o lanche hoje às 12:05.'], 409);
}
```

QUEM BARRA O SEGUNDO LANCHE

O `UNIQUE (aluno_id, data)` da tabela `entregas`. Mesmo que dois atendentes leiam o mesmo QR no mesmo segundo, o banco aceita **só um**.

O modelo por trás de tudo

DER – Banco cantina (modelo de Peter Chen)



Uma turma tem N alunos • um aluno retira N lanches (um por dia) • um usuário registra N entregas. Todas as tabelas em `InnoDB`, para as chaves estrangeiras valerem.

CÓDIGO DO PROJETO

Baixe e rode na sua máquina

REPOSITÓRIO NO GITHUB

github.com/wellingtonsenai/aplicativo

1. Clone dentro do WAMP

```
cd C:\wamp64\www
git clone https://github.com/wellingtonsenai/aplicativo.git
```

2. Crie o banco: importe `api/banco.sql` no phpMyAdmin (aba Importar)

3. Abra por `http://localhost/aplicativo/` — nunca pelo duplo clique (`file:///`)

   Login de teste: supervisor `2001` / `super2026` • atendente `1001` / `cantina123`

Resumo

O navegador mostra, o PHP decide, o MySQL garante.

- ✓ API em PHP devolvendo JSON
- ✓ Senha e regras longe do F12
- ✓ QR com código aleatório
- ✓ 1 lanche por dia pelo UNIQUE